Guía para Andrés · Integrante 1

Evaluación final DGITI · Video de 5 minutos 45 segundos

---

1. Contexto: qué estamos haciendo

El examen final pide **un video de máximo 10 minutos** en el que cada integrante muestra **su propia cuenta de AWS Academy** y hace **en vivo** una parte técnica. Si alguien solo narra o no muestra su cuenta, **baja la nota de los dos**.

El proyecto ya está hecho y está en GitHub: **https://github.com/CristianCamiloChaparro/dgiti-ef**

Es un servicio web en AWS creado con **Terraform**. Tiene una VPC en 2 zonas, un balanceador (ALB), de 2 a 4 servidores EC2 con escalado automático, una base de datos RDS MySQL con réplica, S3, un pipeline de GitHub Actions, una prueba de carga con k6 y un dashboard de CloudWatch.

Los dos tenemos que tener la misma infraestructura desplegada, cada uno en su propia cuenta, con el mismo código.

Qué está haciendo Cristian (Integrante 2)

- Ya armó el proyecto y lo subió a GitHub (es el dueño del repo).
- Ya está desplegando la infraestructura en su cuenta de AWS Academy.
- Va a correr el pipeline de CI/CD y una prueba de carga previa, para que CloudWatch ya muestre el escalado.
- En el video hace **Pipeline CI/CD + Desempeño y monitoreo** (de 2:45 a 5:45, el 55 % de la nota).

Qué haces tú (Integrante 1)

- Despliegas la misma infraestructura **en tu cuenta**: es casi todo esperar, unos 30 minutos.
- En el video haces **Arquitectura + Servicio desplegado + Terraform** (de 0:00 a 2:45, el 45 % de la nota).
- No necesitas tocar GitHub Actions, ni los Secrets, ni k6.

Cómo se graba

Una reunión de Meet, Teams o Zoom grabada, **en una sola toma y sin cortes**. Tú compartes tu pantalla primero y al terminar le pasas a Cristian, que comparte la suya.

---

2. Lo que tienes que hacer HOY, antes de grabar

> **Todos los comandos se escriben en Git Bash**, no en PowerShell ni en CMD. En VS Code: menú **Terminal** → **New Terminal**, luego clic en la flechita ˅ junto al **+** → **Git Bash**.

Paso 0 · Tener las herramientas (si ya las tienes, sáltalo)

Necesitas **Git**, **Terraform**, **AWS CLI** y **VS Code**. En PowerShell:

```
winget install Git.Git Hashicorp.Terraform Amazon.AWSCLI Microsoft.VisualStudioCode
```

Cierra y vuelve a abrir VS Code. Luego comprueba en Git Bash:

```
git --version && terraform version && aws --version
```

Paso 1 · Clonar el proyecto

```
cd ~/Desktop
git clone https://github.com/CristianCamiloChaparro/dgiti-ef.git
cd dgiti-ef
code .
```

Paso 2 · Credenciales de tu AWS Academy

1. Entra a **awsacademy.instructure.com** → **Cursos** → el curso del Learner Lab → **Módulos** → **Launch AWS Academy Learner Lab**.
2. Haz clic en ▶ **Start Lab** y espera a que el círculo **AWS** se ponga **verde**.
3. Haz clic en ⓘ **AWS Details** → junto a *AWS CLI*, clic en **Show** → copia todo el bloque `[default] … aws_session_token=…`.
4. En Git Bash: `bash mkdir -p ~/.aws && code ~/.aws/credentials` **Borra todo** lo que tenga, **pega** el bloque y **guarda** con Ctrl+S.
5. Comprueba: `bash aws sts get-caller-identity` Tiene que mostrar tu número de cuenta y `voclabs`. Si sale `ExpiredToken` o `InvalidClientTokenId`, repite los puntos 3 y 4.

Paso 3 · Desplegar con Terraform (unos 25 minutos)

```
cd terraform
cp terraform.tfvars.example terraform.tfvars
code terraform.tfvars
```

Cambia `db_password` por una clave tuya, **solo letras y números**, de al menos 8 caracteres (por ejemplo `Andres2026Dgiti`). Guarda.

```
terraform init
terraform apply
```

Revisa que diga `Plan: 37 to add`, escribe `yes` y Enter. **Tarda unos 20 a 25 minutos** (lo más lento es la base de datos y su réplica). No cierres la terminal. Al final debe decir `Apply complete! Resources: 37 added`. Entonces:

```
terraform output
cd ..
```

> Si sale un error en rojo, copia el mensaje completo y mándaselo a Cristian. Casi siempre basta con volver a ejecutar `terraform apply`: retoma desde donde quedó.

Si el error dice `Access denied when attempting to assume role ... AWSServiceRoleForAutoScaling` (en una cuenta nueva, AWS a veces crea ese rol unos segundos tarde). El Auto Scaling Group sí quedó creado. Ejecuta esto, en orden, desde la carpeta `terraform`:

```
aws autoscaling suspend-processes --auto-scaling-group-name dgiti-ef-asg --scaling-processes Launch
aws autoscaling resume-processes --auto-scaling-group-name dgiti-ef-asg --scaling-processes Launch
terraform untaint aws_autoscaling_group.app
terraform apply
```

En el `apply`, revisa que diga `0 to destroy` y escribe `yes`.

Paso 4 · Comprobar que funciona

Espera unos **3 minutos** a que arranquen los servidores. Luego, desde la carpeta `dgiti-ef`:

```
./scripts/demo_servicio.sh
```

Tiene que salir: - `/health` → `"status": "ok"` - `/status` → `"ok": true`, con `primary` y `replica` en `"ok": true` - `GET /api/visits` → `"read_from": "replica"`

Además, abre en el navegador la URL que sale como `app_url` en `terraform output` y recarga varias veces: debe cambiar la instancia o la zona. Si la réplica sale con `"ok": false`, espera 5 minutos y vuelve a probar.

Paso 5 · Ensayar tu parte con cronómetro

Lee la sección 3 y ensáyala completa. **Debe durar menos de 2:45.** - En el **ensayo** usa `app_version=1.1.0`. - En la **grabación real** usa `app_version=1.2.0`, porque tiene que ser distinta para que el plan muestre un cambio.

Paso 6 · 15 minutos antes de grabar

- [ ] Learner Lab → **Start Lab** otra vez (renueva las 4 horas de la sesión) → `aws sts get-caller-identity` funciona. Si cambiaron las credenciales, pégalas otra vez (paso 2).
- [ ] Letra grande en la terminal (**Ctrl +**), notificaciones apagadas.
- [ ] Deja abiertas estas pestañas:
- Learner Lab (con el círculo verde)
- Diagrama: **app.diagrams.net** → *Open Existing Diagram* → `docs/arquitectura-aws.drawio`
- Consola AWS → **VPC** → **Subnets**
- Navegador con tu `app_url`
- VS Code con la carpeta `terraform/` a la izquierda y **Git Bash en la carpeta** `dgiti-ef`

---

3. Tu parte del video · 0:00 – 2:45

> Lo que está en *cursiva* es lo que dices. No hace falta decirlo de memoria, pero sí mencionar las palabras en **negrita**.

0:00 – 0:15 · Tu cuenta (15 s)

Muestra: la pestaña del Learner Lab con el círculo **AWS** ● verde. Haz clic en **AWS** para abrir la consola y señala arriba a la derecha `voclabs/user…` y el número de cuenta.

"Soy Andrés. Esta es mi cuenta de AWS Academy: el laboratorio está activo y esta es mi consola, con mi número de cuenta."

0:15 – 1:05 · Arquitectura (50 s)

Muestra: el diagrama.

*"Desplegamos en us-east-1 una **VPC 10.0.0.0/16** en **dos zonas de disponibilidad**, para que si cae una zona el servicio siga. Cada zona tiene **tres subredes**: una pública con el balanceador y el NAT, una privada de aplicación con las EC2 y una privada de base de datos **sin ruta a Internet**.*

*Los usuarios entran por el Internet Gateway a un **Application Load Balancer**, que revisa las instancias con un **health check a /health** y reparte el tráfico. Las EC2 están en un **Auto Scaling Group de 2 a 4 instancias** que escala cuando la **CPU pasa del 50 %** y reemplaza solo cualquier instancia que falle: eso es la **alta disponibilidad**.*

*La seguridad son **tres Security Groups encadenados**: el balanceador recibe el puerto 80 desde Internet, las EC2 solo aceptan el 8080 desde el balanceador y la base de datos solo el 3306 desde las EC2. Ninguna EC2 tiene IP pública. Los datos están en **RDS MySQL con una réplica de lectura** en la otra zona, S3 guarda el código y los reportes del pipeline, y CloudWatch monitorea todo."*

Haz (10 s): pestaña de la consola → **VPC** → **Subnets** → escribe `dgiti-ef` en el filtro. *"Y aquí están las seis subredes del diagrama, creadas en mi cuenta."*

1:05 – 1:35 · Servicio desplegado (30 s)

1. Pestaña del navegador con `app_url`: **recarga 3 veces** (F5). *"Cada recarga la atiende **otra instancia en otra zona**: el balanceador está repartiendo."*

2. En Git Bash:

```
./scripts/demo_servicio.sh
```

*"**/health** es el endpoint de validación del balanceador. **/status** confirma la conexión con la base **primaria y con la réplica**. El **POST** escribe una visita en la primaria y el **GET** la lee desde la réplica: dice `read_from: replica`."*

1:35 – 2:45 · Infraestructura como Código (70 s)

Muestra: VS Code con la carpeta `terraform/` a la izquierda y la terminal abajo.

```
cd terraform
terraform init
terraform plan -var="app_version=1.2.0" -out=tfplan
```

Mientras el plan consulta AWS (tarda unos 30 segundos), haz clic en cada archivo y di:

*"**provider.tf** define el proveedor AWS y la región, **sin credenciales en el código**. **variables.tf** parametriza la red, el cómputo y la base de datos; la contraseña es **sensible** y tiene validación. **main.tf** crea todo: la red, los Security Groups, RDS con su réplica, S3, el balanceador, el Auto Scaling Group y el dashboard de CloudWatch.*

user_data.sh *es lo que ejecuta cada EC2 al arrancar: descarga la app desde S3 y la levanta.* ***outputs.tf*** *entrega la URL y los endpoints. El* ***terraform.tfvars*** *con la contraseña* ***no se sube a Git****; solo el ejemplo. Nada está creado a mano."*

Cuando termine el plan: *"El plan detecta un solo cambio, la* ***versión de la aplicación****: Terraform crea una versión nueva del Launch Template y actualiza el Auto Scaling Group para que las instancias nuevas arranquen con esa versión; el pipeline es el que las reemplaza de una en una, sin cortar el servicio."*

```
terraform apply tfplan
terraform output
```

Al ver **Apply complete!**: *"Aplicado. Y estos son los outputs: la URL del servicio, los endpoints de la base de datos, el bucket y el dashboard. Le doy paso a Cristian."*

→ **Dejas de compartir pantalla y comparte Cristian.**

---

4. Si algo falla mientras grabas

Problema	Qué hacer
`ExpiredToken` o `InvalidClientTokenId`	Las credenciales vencieron: Learner Lab → AWS Details → Show → pégalas de nuevo en `~/.aws/credentials` y repite el comando
El plan muestra más cambios de lo esperado	Di *"Terraform detecta diferencias con el código y las corrige"* y sigue con `terraform apply tfplan`
`Error: Saved plan is stale`	Repite el `terraform plan ... -out=tfplan` y luego el `apply tfplan`
`./scripts/demo_servicio.sh: Permission denied`	Usa `bash scripts/demo_servicio.sh`
El navegador muestra siempre la misma instancia	Recarga con **Ctrl+F5**; si sigue igual, muestra solo el script

5. Preguntas que puede hacer el profesor

Pregunta	Respuesta corta
¿Por qué dos zonas?	Si cae una zona completa, la otra sigue atendiendo
¿Por qué ALB y no NLB?	El ALB trabaja en capa 7: revisa la app con un health check HTTP
¿Por qué las EC2 en subred privada?	No se pueden alcanzar desde Internet; solo entra lo que pasa por el ALB
¿Por qué Security Groups que apuntan a otros SG?	Cada instancia nueva del ASG queda autorizada sola, sin depender de IPs
¿Por qué un solo NAT?	Ahorro de costo en el lab; en producción iría uno por zona
¿Por qué réplica y no Multi-AZ?	La réplica separa lecturas y deja una copia en otra zona; Multi-AZ duplica el costo del lab
¿Dónde está el estado de Terraform?	Local, porque cada uno despliega en su cuenta; en producción iría en S3

6. Después de grabar (obligatorio)

Para no gastar el presupuesto del lab, desde la carpeta `dgiti-ef`:

```
terraform -chdir=terraform destroy
```

Escribe `yes`. Tarda unos 10 a 15 minutos.